

AirSight AI

Air Quality Index Prediction and Monitoring System

Project Report

Submitted as part of the internship requirements

Student Name: Rida Eman
Internship Program: 10Pearls Internship — Cohort 9
Department: Data Science
Project Name: AirSight AI
Date: September 2026

10Pearls Pakistan

Data Science Department

1. Introduction

Air pollution is one of the most critical environmental and public health challenges in Pakistan. Major urban centers such as Lahore and Karachi regularly experience severe smog and high concentrations of toxic particulate matter. These conditions cause serious respiratory illnesses, decrease outdoor visibility, and affect daily life.

Traditional air monitoring systems only display historical or current air quality readings. However, citizens, schools, athletes, and city planners need to know what the air quality will be like in the upcoming hours to plan their activities safely.

AirSight AI is an intelligent air-quality prediction and environmental monitoring platform that I developed during the 10Pearls Cohort 9 Data Science Internship. The system collects atmospheric and meteorological data, trains multiple machine learning models, and generates next-hour predictions as well as 24-hour, 48-hour, and 72-hour forecasts for five major Pakistani cities: Lahore, Karachi, Islamabad, Peshawar, and Quetta.

The platform includes an interactive Streamlit web dashboard where users can view live Air Quality Index (AQI) values, explore geographic station maps, examine 72-hour forecast trends, understand which environmental features drive predictions using Explainable AI (SHAP), and receive practical recommendations on the safest times for outdoor activities.

2. Project Objective

The main objectives of the AirSight AI project are:

1. **Automated Data Collection:** Collect live and historical air pollution data (PM2.5, PM10, CO, NO2, O3, SO2, NH3, NO) and weather data (temperature, humidity, wind speed) from OpenWeather APIs.
2. **Data Preprocessing and Feature Engineering:** Clean raw sensor measurements, convert pollutant concentrations to continuous US EPA AQI values (0–500 scale), and generate time-series lag and rolling-mean features.
3. **Machine Learning Modeling:** Train, evaluate, and compare four machine learning algorithms (Random Forest, Ridge Regression, XGBoost, and Long Short-Term Memory networks) alongside an ensemble model.
4. **Multi-Step Recursive Forecasting:** Build a time-series forecasting engine that predicts AQI values up to 72 hours into the future without data leakage.
5. **Explainable AI (XAI):** Integrate SHAP (SHapley Additive exPlanations) so users can easily see how different weather and pollutant features influence predictions.
6. **Actionable Recommendations:** Create an “Air Quality Windows” engine to identify the best contiguous hours of the day for outdoor activities.
7. **Interactive Dashboard and MLOps:** Build an easy-to-use Streamlit web interface and automate scheduled pipelines using GitHub Actions and the Hopsworks feature store.

3. Understanding of the Project

The AirSight AI project follows a complete end-to-end data science lifecycle. The complete system workflow proceeds through six connected stages:

Data Collection → Data Processing → Feature Engineering → Model Training → Prediction & Forecasting → Dashboard & Alerts

3.1 Pipeline Stages

1. **Data Collection** (`src/data_pipeline/`): Historical data (3,480 raw records) was collected and merged into `data/combined_historical_data.csv`. Live hourly snapshots are fetched from the OpenWeather API using `fetch_data.py`.
2. **Feature Engineering** (`src/feature_engineering/features.py`): Raw data is cleaned, converted to continuous EPA AQI, and transformed into 19 engineered features (including lag and rolling variables), producing `data/processed_features.csv`.
3. **Model Training** (`src/training/train_model.py`): Models are trained on historical data using chronological train/test splits. Trained model artifacts are serialized in the `models/` folder and synced to the Hopsworks model registry.
4. **Inference and Forecasting** (`src/prediction/`): `predict.py` generates next-hour predictions, while `forecast.py` generates 24-hour, 48-hour, and 72-hour recursive forecasts.
5. **Dashboard Visualization** (`src/dashboard/`): The Streamlit dashboard (`app.py`) presents interactive charts, maps, metrics, and SHAP feature attributions.
6. **Alerting System** (`src/alerts/`): `check_alerts.py` checks if predictions exceed the “Unhealthy” threshold and sends email notifications via SMTP.

4. Data Used in the Project

4.1 Monitored Cities

The project covers five major geographic stations across Pakistan:

- **Lahore** (31.5204°N, 74.3587°E) — Major industrial and agricultural plains.
- **Karachi** (24.8607°N, 67.0011°E) — Coastal metropolis and port city.
- **Islamabad** (33.6844°N, 73.0479°E) — Northern capital plateau.
- **Peshawar** (34.0151°N, 71.5249°E) — Northwestern valley.
- **Quetta** (30.1798°N, 66.9750°E) — High-altitude western mountain basin.

4.2 Features and Variables

The dataset contains 19 features used as model inputs:

- **Pollutants (8 variables):** PM_{2.5}, PM₁₀, CO, NO₂, O₃, SO₂, NH₃, and NO (measured in $\mu\text{g}/\text{m}^3$).
- **Meteorology (3 variables):** Temperature (°C), Relative Humidity (%) and Wind Speed (m/s).
- **Time Features (4 variables):** Hour of day (0–23), Day of month (1–31), Month (1–12), and Day of week (0–6).
- **Lag and Rolling Features (4 variables):** 1-hour lag (AQI_{t-1}), 2-hour lag (AQI_{t-2}), 1-hour rate of change ($\text{AQI}_{t-1} - \text{AQI}_{t-2}$), and 3-hour rolling mean.

4.3 Target Variable and EPA AQI Formula

The target variable is `target_aqi`, which represents the continuous AQI value for the next hour ($t + 1$). The continuous AQI is calculated using the official US EPA piecewise linear formula based on PM_{2.5} concentration (C):

$$\text{AQI} = \frac{I_{\text{high}} - I_{\text{low}}}{C_{\text{high}} - C_{\text{low}}}(C - C_{\text{low}}) + I_{\text{low}} \quad (1)$$

where C_{low} and C_{high} are the concentration breakpoints, and I_{low} and I_{high} are the corresponding index breakpoints. The values are categorized into six standard EPA health categories: Good (0–50), Moderate (51–100), Unhealthy for Sensitive Groups (101–150), Unhealthy (151–200), Very Unhealthy (201–300), and Hazardous (301–500).

5. Machine Learning Models

To produce accurate and resilient predictions, AirSight AI uses four different machine learning models:

1. **Random Forest Regressor** (`aqi_random_forest.pkl`): An ensemble of decision trees that handles non-linear relationships well. It also provides the basis for SHAP explainability.
2. **Ridge Regression** (`aqi_ridge.pkl`): A regularized linear regression model with standard scaling to provide a fast and reliable baseline.
3. **XGBoost Regressor** (`aqi_xgboost.pkl`): A gradient-boosted decision tree algorithm that sequentially corrects prediction errors. It achieved the highest individual accuracy in the project.

4. **LSTM Neural Network** (`aqi_lstm.keras`): A deep learning Long Short-Term Memory recurrent neural network that processes 24-hour sequence windows to capture sequential time patterns.
5. **Ensemble Model**: Computes the average prediction across all active models, smoothing individual model errors and providing stable forecasts.

5.1 Model Performance Evaluation

Table 1 presents the evaluation metrics recorded on test data in `models/model_performance.json`.

Table 1: Model Performance Metrics on Test Set

Model	MAE	RMSE	R^2 Score	Validation RMSE
XGBoost	2.34	6.90	0.9642	5.72
Random Forest	2.57	7.09	0.9621	5.94
Ridge Regression	3.40	8.56	0.9448	6.64
LSTM Network	4.79	6.96	0.9646	8.21

6. AQI Forecasting

AirSight AI includes a custom multi-step recursive forecasting algorithm implemented in `src/prediction/forecast.py`.

6.1 Forecasting Mechanism

To predict AQI up to 72 hours ahead without target data leakage:

1. At step $k = 1$, the models use the latest observed features to predict $\widehat{AQI}_{t+1}$.
2. For steps $k = 2, \dots, 72$, the predicted AQI value is recursively fed into the lag queue (`lag1`, `lag2`, rolling mean).
3. Future weather and pollutant values are dynamically fetched from the OpenWeather 5-day forecast API. If the API is unavailable, a diurnal sinusoidal model calculates realistic day and night variations.

6.2 Air Quality Windows Engine

Implemented in `src/recommendation/air_quality_windows.py`, this feature translates raw numbers into practical daily advice. It scores potential outdoor time blocks (0–100) for activities such as Walking, Running, Cycling, or Outdoor Work. It identifies and displays the “Best Outdoor Window” (e.g., 06:00 AM – 09:00 AM) with its expected average AQI and suitability classification.

7. Dashboard

The user interface is built in Streamlit (`src/dashboard/app.py`) and structured into six clear views:

1. **Home / Overview** (`overview.py`): Displays the hero current AQI card, next-hour outlook, five key metric cards (AQI, PM2.5, PM10, Temperature, Humidity), historical trend charts, and an interactive geographic map.
2. **Geographic Map** (`map_view.py`): An interactive station map built with modern Plotly MapLibre (`px.scatter_map`) and OpenStreetMap tiles.
3. **City Details** (`city_intelligence.py`): Detailed pollutant breakdowns and environmental relationships for each city.
4. **Forecast** (`forecasting.py`): 72-hour forecast line charts, multi-model comparison, and the Air Quality Windows recommendation card.
5. **AI Performance & XAI** (`model_intelligence.py`): Evaluation metrics table, model architecture notes, and on-demand SHAP feature attribution tables.
6. **Alerts** (`alerts.py`): Live alert status, customizable notification thresholds, and SMTP email delivery configuration.
7. **Settings** (`settings.py`): User settings saved locally to `data/app_settings.json`.

8. MLOps and Automation

AirSight AI uses automated background pipelines with GitHub Actions and Hopworks:

- **Hourly Feature Pipeline** (`.github/workflows/feature_pipeline.yml`): Runs every hour to fetch live data with `fetch_data.py`, engineer features with `features.py`, check email alerts with `check_alerts.py`, and sync data to the Hopworks feature store.
- **Daily Training Pipeline** (`.github/workflows/training_pipeline.yml`): Runs daily at midnight to retrain models and update model artifacts.
- **Dependency Separation**: To keep the Streamlit dashboard lightweight and fast, runtime dependencies (`requirements.txt`) are separated from MLOps training dependencies (`requirements-mlops.txt`).

9. Problems Faced During Development

During the development of AirSight AI, I encountered several real-world technical problems and resolved them as follows:

9.1 Problem 1: Missing Root Module in GitHub Actions

- **Problem:** The automated feature pipeline in GitHub Actions failed with `ModuleNotFoundError: No module named 'src'`.
- **Why it happened:** When GitHub Actions ran `python src/feature_engineering/features.py`, Python set the working directory to the subfolder rather than the project root.
- **How I handled it:** I added dynamic root path resolution using `pathlib.Path` to `sys.path` in `features.py` and added `env: PYTHONPATH: .` in the GitHub Actions workflow file.

9.2 Problem 2: Deprecated Plotly Mapbox API Crash

- **Problem:** The dashboard crashed on startup with an `AttributeError` inside `map_view.py` at `px.scatter_mapbox`.
- **Why it happened:** Recent Plotly releases (v6.0+) removed the legacy `scatter_mapbox` methods in favor of the MapLibre engine.
- **How I handled it:** I migrated `map_view.py` to modern `px.scatter_map` with `map_style="open-streets"`, eliminating Mapbox token requirements and adding fallback handling.

9.3 Problem 3: Hopsworks and Deep Learning Installation Failures on Streamlit Cloud

- **Problem:** Streamlit Community Cloud deployment failed while attempting to build `h5py` from source and resolving `hopsworks==5.0.6`.
- **Why it happened:** `h5py` required C compilation with system libraries (`pkg-config`, `libhdf5-dev`) not present in the cloud container, while Hopsworks brought heavy dependencies not needed by the dashboard runtime.
- **How I handled it:** I decoupled the requirements into `requirements.txt` (inference only) and `requirements-mlops.txt` (training only), and wrapped Hopsworks and TensorFlow imports in safe `try/except` fallback blocks.

9.4 Problem 4: Preventing Data Leakage in 72-Hour Forecasting

- **Problem:** Forecasting future hours required lag features that did not exist yet for future timestamps.
- **Why it happened:** In time-series forecasting, using future actual values causes target leakage.
- **How I handled it:** I built an autoregressive feedback loop where each hour's predicted AQI is appended to a state buffer to dynamically generate lags for subsequent forecast steps.

9.5 Problem 5: Preventing Duplicate Alert Emails

- **Problem:** The hourly alerting daemon repeatedly sent duplicate emails every hour while the AQI remained in the “Unhealthy” range.
- **Why it happened:** The pipeline had no memory of previous alert states between hourly runs.
- **How I handled it:** I created a persistent state tracking file (`data/alert_state.json`) with file locking to ensure emails are only sent on initial state transitions.

10. Deployment Challenges

Deploying a multi-model machine learning application to Streamlit Community Cloud presented specific deployment constraints:

1. **Monolithic Dependency Bloat:** The initial repository used a full local environment freeze containing Jupyter, debugging tools, and build packages (160+ dependencies). This caused dependency resolution conflicts and slow build times.
2. **Binary Wheel Compatibility:** Cloud Linux containers require pre-compiled manylinux wheels. By removing strict version pins and training-only packages from `requirements.txt`, container build time dropped from over 10 minutes to under 25 seconds.
3. **Cross-Platform File Paths:** Local Windows paths (e.g., backslashes `\`) caused path resolution errors in Linux containers. All file paths were refactored to use `pathlib.Path` anchored to the project root.

11. Testing and Validation

AirSight AI includes an automated test suite located in the `tests/` directory. Testing was executed using `pytest`:

- `test_air_quality_windows.py`: Validates suitability score calculations (0–100), activity profile penalties, and candidate time window ranking (7 tests).
- `test_alerts.py`: Verifies threshold categorization, email formatting, and duplicate alert suppression via state locking (6 tests).
- `test_feature_view.py`: Tests feature column schemas, chronological ordering, and train/test split discipline (4 tests).
- `test_local_forecast.py`: Verifies local model loading latency, 72-hour recursive forecast shape, missing file exceptions, and lazy SHAP calculations (6 tests).
- `test_lstm.py`: Validates 24-step sequence generation, absence of future leakage in sequences, and scaling isolation (7 tests).

Test Execution Result: 30 passed out of 30 tests in 29.57 seconds (100% pass rate).

12. What I Learned

Working on AirSight AI during the 10Pearls internship was a comprehensive learning experience that bridged academic data science concepts with production engineering:

- **End-to-End Machine Learning:** I learned how to move from raw, noisy API data to cleaned features, model training, evaluation, and serialized deployment.
- **Time-Series Forecasting:** I gained a deep understanding of autoregressive lagging, rolling window statistics, and how to prevent future target leakage.
- **Production MLOps:** I learned how to use GitHub Actions for scheduled workflows and how to manage feature stores with Hopsworks.
- **Software Engineering Discipline:** I learned the importance of automated unit testing, defensive exception handling, path portability, and separating production runtime requirements from development dependencies.
- **User Interface Design:** Building the dashboard in Streamlit taught me how to present complex predictions and SHAP explainability in a clear, user-friendly manner.

13. Conclusion

AirSight AI successfully demonstrates an end-to-end air quality prediction and environmental intelligence system tailored for major Pakistani cities. By combining tabular gradient boosting (XGBoost, Random Forest), linear modeling (Ridge), deep learning (LSTM), and an ensemble approach, the system achieves strong predictive accuracy ($R^2 > 0.96$).

The project delivers practical value through its 72-hour forecasting engine, smart Air Quality Windows recommendations, on-demand Explainable AI, and automated MLOps pipelines. Developing this project provided valuable hands-on experience in machine learning, system architecture, debugging, and production deployment.